PYTHON ACTIVITY-BASED LEARNING

10 Case Studies • Algorithms • Flowcharts • Pseudocode • Python Programs

PROJECT REPORT	INTRODUCTORY PYTHON
10 selected case studies	Syllabus-aligned techniques
Readable source code	Test cases and sample outputs

Project Overview

The activity sheet requires ten selected case studies. Each case study should include a problem statement, objectives, assumptions, algorithm, flowchart, pseudocode, Python source code, test cases, output screenshots, and conclusion. This report supplies those written components and text-based flowcharts; the accompanying .py files are runnable programs.

Selected Case Studies

1. University Scholarship Allocation
2. Progressive Electricity Billing
3. Employee Payroll and Deduction Analysis
4. Retail Billing with Multiple Discount Rules
5. Attendance Recovery Computation
6. ATM Transaction Sequence Simulation
7. Student Marks Matrix Analysis
8. Tuple-Based Product Catalogue Audit
9. Campus Event Financial Planning
10. University Admission Decision Engine

Syllabus coverage

Variables, data types, arithmetic and comparison operators, conditional statements, for/while loops, strings, lists, nested lists, tuples, tuple unpacking, nested-loop comparisons, and formatted output are used across the selected programs. The solutions avoid external libraries.

Important note

Thresholds, rates, fees, and records are illustrative assumptions made to turn the case studies into complete runnable examples. They are not presented as official real-world policies.

CASE STUDY 01: University Scholarship Allocation

1. Problem Statement

Create a program that evaluates scholarship applicants using aggregate marks, subject-wise minimum scores, attendance, and family-income criteria. Eligible students are ranked and ties are resolved by predefined rules.

2. Objectives

• Check subject-wise minimum marks.

• Calculate aggregate percentage and validate attendance and income limits.

• Separate eligible and ineligible applicants.

• Rank eligible applicants using consistent tie-breakers.

3. Assumptions

• Subjects are Mathematics, Physics, and Computer Science; each is marked out of 100.

• Each subject must be at least 50; aggregate must be at least 75%.

• Attendance must be at least 85%; annual family income must not exceed ■250,000.

• Ties are resolved by higher aggregate, then higher Mathematics mark, then alphabetical name.

4. Algorithm

1. Store applicant records as tuples in a list.

2. For each applicant, unpack the tuple and calculate aggregate percentage.

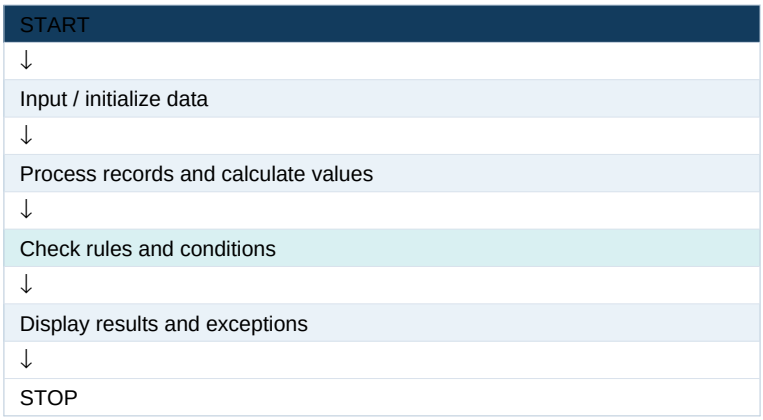
3. Check all subject, aggregate, attendance, and income conditions.

4. Add qualifying records to the eligible list; otherwise record the reason for ineligibility.

5. Sort eligible applicants using nested loops and the tie-breaking rules.

6. Display ranked applicants and ineligible applicants with reasons.

5. Flowchart



6. Pseudocode

```
START
Set subject minimum = 50, aggregate minimum = 75, attendance minimum = 85, income maximum = 250000
Read applicant records
FOR each applicant: calculate aggregate; check every eligibility condition
IF all conditions pass: add to eligible list ELSE: record failed conditions
Rank eligible records by aggregate descending, Mathematics descending, name ascending
Display eligible ranking and ineligible reasons
STOP
```

7. Python Source Code

```
# Case Study 1: University Scholarship Allocation
MIN_SUBJECT = 50
MIN_AGGREGATE = 75
MIN_ATTENDANCE = 85
MAX_INCOME = 250000
```

```
# (name, maths, physics, computer_science, attendance, annual_income)
applicants = [
    ("Anil", 88, 82, 91, 94, 180000),
    ("Bhavya", 78, 80, 82, 90, 220000),
    ("Charan", 92, 76, 88, 86, 240000),
    ("Divya", 81, 79, 84, 82, 150000),
    ("Esha", 48, 90, 85, 96, 100000),
    ("Farhan", 78, 80, 82, 90, 220000),
]

eligible = []
ineligible = []

for record in applicants:
    name, maths, physics, computer, attendance, income = record
    aggregate = (maths + physics + computer) / 3
    reasons = []

    if maths < MIN_SUBJECT or physics < MIN_SUBJECT or computer < MIN_SUBJECT:
        reasons.append("subject mark below 50")
    if aggregate < MIN_AGGREGATE:
        reasons.append("aggregate below 75%")
    if attendance < MIN_ATTENDANCE:
        reasons.append("attendance below 85%")
    if income > MAX_INCOME:
        reasons.append("income above Rs. 250000")

    if len(reasons) == 0:
        eligible.append((name, maths, physics, computer, attendance, income, aggregate))
    else:
        ineligible.append((name, aggregate, reasons))

# Nested-loop ranking: aggregate desc, maths desc, name asc
for i in range(len(eligible)):
    for j in range(i + 1, len(eligible)):
        a, b = eligible[i], eligible[j]
        swap = False
        if a[6] < b[6]:
            swap = True
        elif a[6] == b[6] and a[1] < b[1]:
            swap = True
        elif a[6] == b[6] and a[1] == b[1] and a[0] > b[0]:
            swap = True
        if swap:
            eligible[i], eligible[j] = eligible[j], eligible[i]

print("UNIVERSITY SCHOLARSHIP ALLOCATION")
print("=" * 38)
print("ELIGIBLE APPLICANTS")
for rank, record in enumerate(eligible, start=1):
    name, maths, physics, computer, attendance, income, aggregate = record
    print(f"{rank}. {name:10} Aggregate: {aggregate:.2f}% | Attendance: {attendance}%")
print("\nINELIGIBLE APPLICANTS")
for name, aggregate, reasons in ineligible:
    print(f"{name:10} Aggregate: {aggregate:.2f}% | Reason: {', '.join(reasons)}")
```

8. Test Cases

Test / input	Condition	Expected result
Anil	Marks 88,82,91; attendance 94; income 180000	Eligible; aggregate 87.00%
Divya	Attendance 82%	Ineligible: attendance below 85%
Esha	Mathematics 48	Ineligible: subject mark below 50
Bhavya and Farhan	Same aggregate and Mathematics marks	Alphabetical tie-break: Bhavya before Farhan

9. Sample Output

```
UNIVERSITY SCHOLARSHIP ALLOCATION
======================================
ELIGIBLE APPLICANTS
1. Anil       Aggregate: 87.00% | Attendance: 94%
2. Charan     Aggregate: 85.33% | Attendance: 86%
3. Bhavya     Aggregate: 80.00% | Attendance: 90%
4. Farhan     Aggregate: 80.00% | Attendance: 90%

INELIGIBLE APPLICANTS
Divya      Aggregate: 81.33% | Reason: attendance below 85%
Esha       Aggregate: 74.33% | Reason: subject mark below 50, aggregate below 75%
```

10. Conclusion

The program applies multiple eligibility constraints, explains failed conditions, and ranks qualifying applicants deterministically. It demonstrates variables, arithmetic and comparison operators, conditionals, loops, strings, lists, tuples,

unpacking, and nested-loop ranking.

CASE STUDY 02: Progressive Electricity Billing

1. Problem Statement

Calculate household electricity bills using progressive consumption slabs, fixed charges, surcharge, and rebates. Compare monthly bills and flag unusually large increases.

2. Objectives

• Calculate charges for each consumption slab.

• Apply fixed charges, surcharge, and rebate rules.

• Compare two months and detect an unusual increase.

3. Assumptions

• First 100 units cost ■2.00/unit; next 100 cost ■3.00/unit; units above 200 cost ■5.00/unit.

• Fixed charge is ■75 per month.

• A 5% surcharge applies when consumption exceeds 250 units.

• A ■50 rebate applies when consumption is at most 100 units.

• An increase above 30% from the previous month is flagged.

4. Algorithm

1. Store monthly unit readings in a list of tuples.

2. Calculate slab charges according to consumption.

3. Add fixed charge and applicable surcharge; subtract any rebate.

4. Compare each bill with the previous month's bill.

5. Display itemized monthly totals and any unusual increase.

5. Flowchart

START
↓
Input / initialize data
↓
Process records and calculate values
↓
Check rules and conditions
↓
Display results and exceptions
↓
STOP

6. Pseudocode

```
START
Store monthly units
FOR each month: calculate first-slab, second-slab and remaining-unit charges
Add fixed fee; apply surcharge if units > 250; apply rebate if units <= 100
Compare bill with previous month; flag increase > 30%
Display monthly bill report
STOP
```

7. Python Source Code

```
# Case Study 2: Progressive Electricity Billing
FIXED_CHARGE = 75
SURCHARGE_RATE = 0.05
REBATE = 50
INCREASE_LIMIT = 30

# (month, units_consumed)
monthly_usage = [
    ("January", 95),
```

```
    ("February", 140),
    ("March", 275),
    ("April", 360),
]

def energy_charge(units):
    first_slab = min(units, 100) * 2
    second_slab = min(max(units - 100, 0), 100) * 3
    third_slab = max(units - 200, 0) * 5
    return first_slab + second_slab + third_slab

bills = []
for month, units in monthly_usage:
    charge = energy_charge(units)
    subtotal = charge + FIXED_CHARGE
    surcharge = subtotal * SURCHARGE_RATE if units > 250 else 0
    rebate = REBATE if units <= 100 else 0
    total = subtotal + surcharge - rebate
    bills.append((month, units, charge, surcharge, rebate, total))

print("PROGRESSIVE ELECTRICITY BILL")
print("=" * 35)
for i, bill in enumerate(bills):
    month, units, charge, surcharge, rebate, total = bill
    print(f"{month:10} Units: {units:3} | Energy: Rs.{charge:7.2f} | "
          f"Fixed: Rs.{FIXED_CHARGE:.2f} | Surcharge: Rs.{surcharge:.2f} | "
          f"Rebate: Rs.{rebate:.2f} | Total: Rs.{total:.2f}")
    if i > 0:
        previous_total = bills[i - 1][5]
        if previous_total > 0:
            increase = (total - previous_total) / previous_total * 100
            if increase > INCREASE_LIMIT:
                print(f"  ALERT: bill increased by {increase:.1f}% from previous month.")
```

8. Test Cases

Test / input	Condition	Expected result
January	95 units	Rebate applied; total bill ■215.00
February	140 units	Energy charge ■320.00; total ■395.00
March	275 units	5% surcharge applied
Month comparison	Bill rises by more than 30%	An alert is printed

9. Sample Output

```
PROGRESSIVE ELECTRICITY BILL
==================================
January    Units:  95 | Energy: Rs. 190.00 | Fixed: Rs.75.00 | Surcharge: Rs.0.00 | Rebate: Rs.50.00 | Total: Rs.215.00
February   Units: 140 | Energy: Rs. 320.00 | Fixed: Rs.75.00 | Surcharge: Rs.0.00 | Rebate: Rs.0.00 | Total: Rs.395.00
  ALERT: bill increased by 83.7% from previous month.
March      Units: 275 | Energy: Rs. 875.00 | Fixed: Rs.75.00 | Surcharge: Rs.47.50 | Rebate: Rs.0.00 | Total: Rs.997.50
  ALERT: bill increased by 152.5% from previous month.
April      Units: 360 | Energy: Rs.1300.00 | Fixed: Rs.75.00 | Surcharge: Rs.68.75 | Rebate: Rs.0.00 | Total: Rs.1443.75
  ALERT: bill increased by 44.7% from previous month.
```

10. Conclusion

The program models progressive tariffs, conditional fees, rebates, and month-to-month comparisons. It demonstrates arithmetic, conditionals, loops, functions, lists, tuples, and numeric comparisons.

CASE STUDY 03: Employee Payroll and Deduction Analysis

1. Problem Statement

Calculate employee gross salary from basic pay, allowances, overtime, and incentives, then apply progressive deductions and print a formatted salary statement.

2. Objectives

• Calculate allowances, overtime, incentives, and gross pay.

• Apply a progressive deduction policy.

• Calculate net salary and print a readable payslip.

3. Assumptions

• HRA is 20% of basic pay and other allowance is 10%.

• Overtime is paid at 1.5 times the hourly basic rate; hourly basic rate = basic pay / 160.

• Incentives are added to gross pay.

• Deduction is 5% of gross up to ■25,000, 10% on the next ■25,000, and 15% above ■50,000.

4. Algorithm

1. Store employee details as tuples.

2. Calculate HRA, other allowance, overtime pay, and gross salary.

3. Calculate deductions using progressive ranges.

4. Subtract deductions to obtain net salary.

5. Print each employee's salary statement.

5. Flowchart

START
↓
Input / initialize data
↓
Process records and calculate values
↓
Check rules and conditions
↓
Display results and exceptions
↓
STOP

6. Pseudocode

```
START
FOR each employee record: unpack values
Calculate HRA = 20% basic; other allowance = 10% basic
Calculate overtime pay and gross salary
Calculate progressive deduction across the three ranges
Net salary = gross - deduction
Print payslip
STOP
```

7. Python Source Code

```
# Case Study 3: Employee Payroll and Deduction Analysis

employees = [
    ("Asha", 30000, 8, 2500),
    ("Rahul", 22000, 5, 1200),
    ("Meera", 52000, 10, 5000),
]

def progressive_deduction(gross):
    deduction = min(gross, 25000) * 0.05
```

```
    deduction += min(max(gross - 25000, 0), 25000) * 0.10
    deduction += max(gross - 50000, 0) * 0.15
    return deduction

print("EMPLOYEE PAYROLL STATEMENT")
print("=" * 42)
for name, basic, overtime_hours, incentive in employees:
    hra = basic * 0.20
    other_allowance = basic * 0.10
    hourly_rate = basic / 160
    overtime_pay = overtime_hours * hourly_rate * 1.5
    gross = basic + hra + other_allowance + overtime_pay + incentive
    deduction = progressive_deduction(gross)
    net = gross - deduction

    print(f"Employee: {name}")
    print(f"  Basic pay       : Rs.{basic:,.2f}")
    print(f"  HRA             : Rs.{hra:,.2f}")
    print(f"  Other allowance : Rs.{other_allowance:,.2f}")
    print(f"  Overtime        : Rs.{overtime_pay:,.2f}")
    print(f"  Incentive       : Rs.{incentive:,.2f}")
    print(f"  Gross salary    : Rs.{gross:,.2f}")
    print(f"  Deduction       : Rs.{deduction:,.2f}")
    print(f"  NET SALARY      : Rs.{net:,.2f}")
    print("-" * 42)
```

8. Test Cases

Test / input	Condition	Expected result
Asha	Basic 30000, OT 8 hours, incentive 2500	Gross ■43,750.00; deduction ■3,000.00; net ■40,750.00
Rahul	Basic 22000, OT 5 hours, incentive 1200	Net salary is below gross salary
Meera	Basic 52000, OT 10 hours, incentive 5000	Progressive deduction includes 15% portion above ■50,000

9. Sample Output

```
EMPLOYEE PAYROLL STATEMENT
==========================================
Employee: Asha
  Basic pay       : Rs.30,000.00
  HRA             : Rs.6,000.00
  Other allowance : Rs.3,000.00
  Overtime        : Rs.2,250.00
  Incentive       : Rs.2,500.00
  Gross salary    : Rs.43,750.00
  Deduction       : Rs.3,125.00
  NET SALARY      : Rs.40,625.00
------------------------------------------
Employee: Rahul
  Basic pay       : Rs.22,000.00
  HRA             : Rs.4,400.00
  Other allowance : Rs.2,200.00
  Overtime        : Rs.1,031.25
  Incentive       : Rs.1,200.00
  Gross salary    : Rs.30,831.25
  Deduction       : Rs.1,833.12
  NET SALARY      : Rs.28,998.12
------------------------------------------
Employee: Meera
  Basic pay       : Rs.52,000.00
  HRA             : Rs.10,400.00
  Other allowance : Rs.5,200.00
  Overtime        : Rs.4,875.00
  Incentive       : Rs.5,000.00
  Gross salary    : Rs.77,475.00
  Deduction       : Rs.7,871.25
  NET SALARY      : Rs.69,603.75
------------------------------------------
```

10. Conclusion

The payroll system calculates components of gross pay, progressive deductions, and net salary. It demonstrates arithmetic operators, conditionals, loops, tuples, unpacking, and formatted strings.

CASE STUDY 04: Retail Billing with Multiple Discount Rules

1. Problem Statement

Create an itemized retail invoice that calculates product totals, applies category-specific tax, threshold discounts, and a maximum discount limit.

2. Objectives

• Calculate line totals for multiple products.

• Apply discount thresholds and a discount cap.

• Apply category-specific tax and print an invoice.

3. Assumptions

• Electronics tax is 18%; books tax is 5%; clothing tax is 12%.

• A 10% discount applies when the pre-tax subtotal is at least ■5,000.

• A 5% discount applies when the pre-tax subtotal is at least ■2,500 but below ■5,000.

• Maximum discount is ■1,000.

• Tax is calculated on the subtotal after discount, using each product's category rate.

4. Algorithm

1. Store products as tuples of name, category, unit price, and quantity.

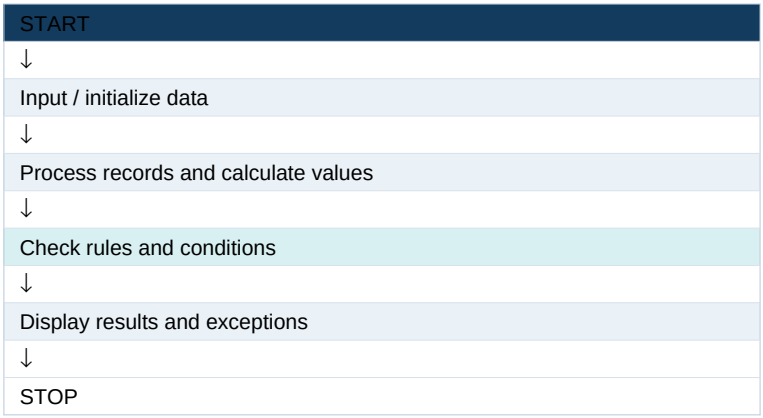
2. Calculate each line total and subtotal.

3. Choose a discount rate based on the subtotal and cap the discount at ■1,000.

4. Calculate category-wise tax on discounted line amounts.

5. Display itemized totals, discount, tax, and final amount.

5. Flowchart



6. Pseudocode

```
START
Store product tuples
Calculate line totals and subtotal
IF subtotal >= 5000: discount rate = 10% ELSE IF subtotal >= 2500: rate = 5% ELSE rate = 0%
Discount = minimum(subtotal × rate, 1000)
Calculate category-wise tax after proportionally applying discount
Final = subtotal - discount + tax
Print invoice
STOP
```

7. Python Source Code

```
# Case Study 4: Retail Billing with Multiple Discount Rules
TAX_RATES = {"electronics": 0.18, "books": 0.05, "clothing": 0.12}
MAX_DISCOUNT = 1000

# (product, category, unit_price, quantity)
cart = [
    ("Headphones", "electronics", 1800, 2),
```

```
    ("Python Book", "books", 650, 2),
    ("T-shirt", "clothing", 500, 1),
]

subtotal = 0
for name, category, price, quantity in cart:
    subtotal += price * quantity

if subtotal >= 5000:
    discount_rate = 0.10
elif subtotal >= 2500:
    discount_rate = 0.05
else:
    discount_rate = 0

discount = min(subtotal * discount_rate, MAX_DISCOUNT)
discount_factor = (subtotal - discount) / subtotal if subtotal else 0
tax_total = 0

print("RETAIL INVOICE")
print("=" * 55)
print(f"{'Product':18} {'Qty':>4} {'Amount':>10} {'Tax':>8}")
for name, category, price, quantity in cart:
    line_total = price * quantity
    discounted_line = line_total * discount_factor
    tax = discounted_line * TAX_RATES[category]
    tax_total += tax
    print(f"{name:18} {quantity:>4} {line_total:>10.2f} {tax:>8.2f}")

final_total = subtotal - discount + tax_total
print("-" * 55)
print(f"Subtotal        : Rs.{subtotal:.2f}")
print(f"Discount        : Rs.{discount:.2f}")
print(f"Tax total       : Rs.{tax_total:.2f}")
print(f"FINAL AMOUNT    : Rs.{final_total:.2f}")
```

8. Test Cases

Test / input	Condition	Expected result
Cart subtotal ■5,000 or more	Discount threshold crossed	10% discount, capped at ■1,000
Cart subtotal ■2,500–■4,999.99	Mid-level threshold crossed	5% discount
Books item	Tax category books	5% tax rate used
Empty cart	Subtotal zero	No division error; final amount ■0.00

9. Sample Output

```
RETAIL INVOICE
=======================================================
Product             Qty     Amount      Tax
Headphones            2    3600.00   583.20
Python Book           2    1300.00    58.50
T-shirt               1     500.00    54.00
-------------------------------------------------------
Subtotal        : Rs.5400.00
Discount        : Rs.540.00
Tax total       : Rs.695.70
FINAL AMOUNT    : Rs.5555.70
```

10. Conclusion

The invoice program combines tuple-based product records, loops, conditional discount rules, category tax rates, and formatted output to produce a clear customer bill.

CASE STUDY 05: Attendance Recovery Computation

1. Problem Statement

Calculate a student's current attendance percentage and determine the minimum consecutive classes that must be attended to reach a required eligibility threshold.

2. Objectives

• Calculate current attendance percentage.

• Find the minimum additional attended classes needed.

• Compare the result against an attendance requirement.

3. Assumptions

• Attendance percentage = attended classes / conducted classes × 100.

• Future classes are all attended in the recovery calculation.

• Required attendance is 75%.

• If no classes have been conducted, the percentage is reported as not yet available.

4. Algorithm

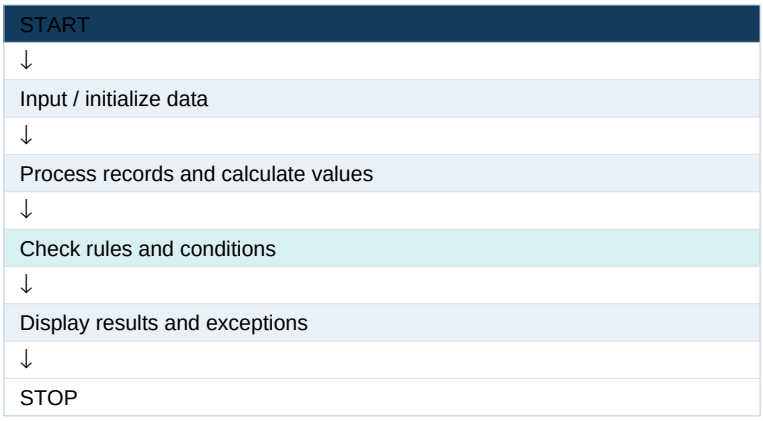
1. Store student attendance records as tuples.

2. Calculate each student's current percentage.

3. Use a loop to add one attended class at a time until the target percentage is reached.

4. Display current percentage and minimum classes required.

5. Flowchart



6. Pseudocode

```
START
Set required attendance = 75%
FOR each student: calculate current attendance percentage
Set extra = 0
WHILE (attended + extra) / (conducted + extra) × 100 < 75: extra = extra + 1
Display current percentage and extra classes
STOP
```

7. Python Source Code

```
# Case Study 5: Attendance Recovery Computation
REQUIRED_PERCENTAGE = 75

# (student_name, classes_attended, classes_conducted)
students = [
    ("Kiran", 42, 60),
    ("Nisha", 35, 50),
    ("Arjun", 28, 40),
    ("Sana", 18, 30),
    ("Dev", 0, 0),
]

print("ATTENDANCE RECOVERY REPORT")
print("=" * 38)
```

```
for name, attended, conducted in students:
    if conducted == 0:
        print(f"{name:10} Current attendance: Not available (no classes conducted)")
        continue

    current = attended / conducted * 100
    extra = 0
    while (attended + extra) / (conducted + extra) * 100 < REQUIRED_PERCENTAGE:
        extra += 1

    print(f"{name:10} Current: {current:.2f}% | Additional consecutive classes: {extra}")
```

8. Test Cases

Test / input	Condition	Expected result
42 attended of 60	Current 70%	Minimum 12 additional classes
35 attended of 50	Current 70%	Minimum 10 additional classes
45 attended of 60	Current 75%	0 additional classes
0 conducted	No attendance history	Report unavailable without division by zero

9. Sample Output

```
ATTENDANCE RECOVERY REPORT
======================================
Kiran      Current: 70.00% | Additional consecutive classes: 12
Nisha      Current: 70.00% | Additional consecutive classes: 10
Arjun      Current: 70.00% | Additional consecutive classes: 8
Sana       Current: 60.00% | Additional consecutive classes: 18
Dev        Current attendance: Not available (no classes conducted)
```

10. Conclusion

The program calculates attendance and uses a while loop to find the minimum future classes required to reach a target. It demonstrates arithmetic, conditions, loops, tuples, unpacking, and edge-case handling.

CASE STUDY 06: ATM Transaction Sequence Simulation

1. Problem Statement

Simulate deposits and withdrawals, maintain an account balance, enforce withdrawal limits, reject invalid transactions, and produce a transaction summary.

2. Objectives

• Process a sequence of deposits and withdrawals.

• Reject invalid amounts and withdrawals beyond balance or limits.

• Summarize accepted and rejected transactions.

3. Assumptions

• Opening balance is ■10,000.

• Each withdrawal must be positive, no greater than ■5,000, and no greater than the current balance.

• Deposits must be positive.

• Transactions are supplied as tuples containing type and amount.

4. Algorithm

1. Set the opening balance and withdrawal limit.

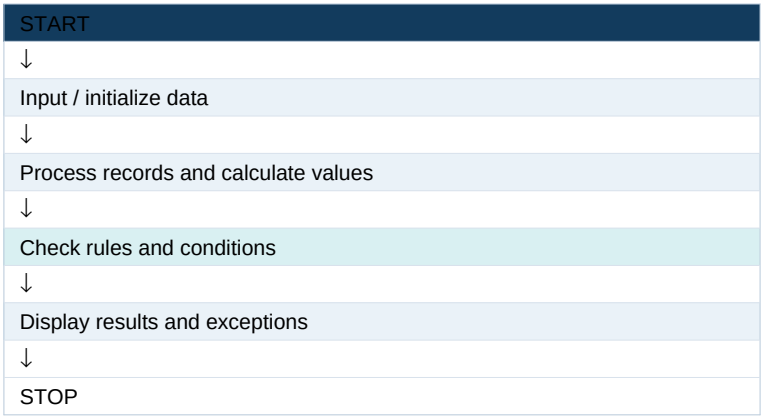
2. Process each transaction tuple in sequence.

3. Validate the amount and transaction type.

4. Update balance for accepted transactions; record rejected transactions.

5. Display transaction outcomes and final totals.

5. Flowchart



6. Pseudocode

```
START
Set opening balance = 10000 and withdrawal limit = 5000
FOR each transaction: unpack type and amount
IF amount <= 0: reject
ELSE IF deposit: add amount
ELSE IF withdrawal and amount <= limit and amount <= balance: subtract amount
ELSE reject with reason
Display accepted/rejected counts, totals, and final balance
STOP
```

7. Python Source Code

```
# Case Study 6: ATM Transaction Sequence Simulation
opening_balance = 10000
withdrawal_limit = 5000

# (transaction_type, amount)
transactions = [
    ("deposit", 2500),
    ("withdrawal", 3000),
    ("withdrawal", 6000),
```

```
    ("deposit", 0),
    ("withdrawal", 2000),
    ("transfer", 500),
]

balance = opening_balance
accepted = 0
rejected = 0
total_deposits = 0
total_withdrawals = 0

print("ATM TRANSACTION SUMMARY")
print("=" * 38)
for transaction_type, amount in transactions:
    if amount <= 0:
        print(f"REJECTED: {transaction_type} Rs.{amount:.2f} (amount must be positive)")
        rejected += 1
    elif transaction_type == "deposit":
        balance += amount
        total_deposits += amount
        accepted += 1
        print(f"ACCEPTED: Deposit Rs.{amount:.2f} | Balance Rs.{balance:.2f}")
    elif transaction_type == "withdrawal":
        if amount > withdrawal_limit:
            print(f"REJECTED: Withdrawal Rs.{amount:.2f} (limit Rs.{withdrawal_limit:.2f})")
            rejected += 1
        elif amount > balance:
            print(f"REJECTED: Withdrawal Rs.{amount:.2f} (insufficient balance)")
            rejected += 1
        else:
            balance -= amount
            total_withdrawals += amount
            accepted += 1
            print(f"ACCEPTED: Withdrawal Rs.{amount:.2f} | Balance Rs.{balance:.2f}")
    else:
        print(f"REJECTED: Unknown transaction type '{transaction_type}'")
        rejected += 1

print("-" * 38)
print(f"Opening balance : Rs.{opening_balance:.2f}")
print(f"Total deposits  : Rs.{total_deposits:.2f}")
print(f"Total withdrawals: Rs.{total_withdrawals:.2f}")
print(f"Accepted        : {accepted}")
print(f"Rejected        : {rejected}")
print(f"FINAL BALANCE   : Rs.{balance:.2f}")
```

8. Test Cases

Test / input	Condition	Expected result
Deposit ■2,500	Positive deposit	Accepted; balance ■12,500
Withdraw ■6,000	Exceeds per-transaction limit	Rejected
Deposit ■0	Non-positive amount	Rejected
Unknown transfer type	Unsupported operation	Rejected and reported

9. Sample Output

```
ATM TRANSACTION SUMMARY
======================================
ACCEPTED: Deposit Rs.2500.00 | Balance Rs.12500.00
ACCEPTED: Withdrawal Rs.3000.00 | Balance Rs.9500.00
REJECTED: Withdrawal Rs.6000.00 (limit Rs.5000.00)
REJECTED: deposit Rs.0.00 (amount must be positive)
ACCEPTED: Withdrawal Rs.2000.00 | Balance Rs.7500.00
REJECTED: Unknown transaction type 'transfer'
--------------------------------------
Opening balance : Rs.10000.00
Total deposits  : Rs.2500.00
Total withdrawals: Rs.5000.00
Accepted        : 3
Rejected        : 3
FINAL BALANCE   : Rs.7500.00
```

10. Conclusion

The ATM simulation processes transaction sequences safely, updates balances only for valid operations, and provides an audit summary. It demonstrates loops, conditions, tuples, unpacking, counters, and validation.

CASE STUDY 07: Student Marks Matrix Analysis

1. Problem Statement

Analyze a nested list of student marks across subjects, calculate student totals and averages, find top performers, and identify subjects with low average performance.

2. Objectives

• Calculate row-wise totals and averages.

• Calculate subject-wise averages from columns.

• Identify the highest-scoring student and low-performing subjects.

3. Assumptions

• Each mark is out of 100.

• All students have marks for the same subjects.

• A subject average below 60 is considered low.

• Student and subject lists match the dimensions of the marks matrix.

4. Algorithm

1. Store student names, subject names, and marks as a nested list.

2. Calculate each student's total and average.

3. Calculate each subject's average using nested loops.

4. Find the top student and subjects below the threshold.

5. Print a performance report.

5. Flowchart

START
↓
Input / initialize data
↓
Process records and calculate values
↓
Check rules and conditions
↓
Display results and exceptions
↓
STOP

6. Pseudocode

```
START
Store names, subjects, and marks matrix
FOR each row: calculate total and average
FOR each subject column: sum marks and calculate average
Find maximum student average
List subjects with average < 60
Display report
STOP
```

7. Python Source Code

```
# Case Study 7: Student Marks Matrix Analysis
students = ["Aarav", "Diya", "Ishaan", "Meera"]
subjects = ["Python", "Mathematics", "Communication"]
marks = [
    [88, 79, 91],
    [76, 85, 82],
    [54, 61, 58],
    [93, 89, 95],
]
LOW_AVERAGE_LIMIT = 60
```

```
print("STUDENT PERFORMANCE ANALYSIS")
print("=" * 44)
student_averages = []
for i in range(len(students)):
    total = 0
    for j in range(len(subjects)):
        total += marks[i][j]
    average = total / len(subjects)
    student_averages.append(average)
    print(f"{students[i]:10} Total: {total:3} | Average: {average:.2f}%")

top_index = 0
for i in range(1, len(student_averages)):
    if student_averages[i] > student_averages[top_index]:
        top_index = i
print(f"\nTop performer: {students[top_index]} ({student_averages[top_index]:.2f}%)")

print("\nSUBJECT-WISE AVERAGES")
for j in range(len(subjects)):
    total = 0
    for i in range(len(students)):
        total += marks[i][j]
    average = total / len(students)
    status = "LOW AVERAGE" if average < LOW_AVERAGE_LIMIT else "Satisfactory"
    print(f"{subjects[j]:18} Average: {average:.2f}% | {status}")
```

8. Test Cases

Test / input	Condition	Expected result
Meera	Marks 93,89,95	Top performer; average 92.33%
Ishaan	Marks 54,61,58	Student average 57.67%
Python column	88,76,54,93	Subject average 77.75%; satisfactory
Matrix dimensions	4 students × 3 subjects	All row and column calculations complete

9. Sample Output

```
STUDENT PERFORMANCE ANALYSIS
============================================
Aarav      Total: 258 | Average: 86.00%
Diya       Total: 243 | Average: 81.00%
Ishaan     Total: 173 | Average: 57.67%
Meera      Total: 277 | Average: 92.33%

Top performer: Meera (92.33%)

SUBJECT-WISE AVERAGES
Python             Average: 77.75% | Satisfactory
Mathematics        Average: 78.50% | Satisfactory
Communication      Average: 81.50% | Satisfactory
```

10. Conclusion

The matrix analysis calculates student and subject statistics with nested loops and nested lists. It demonstrates indexing, accumulation, comparisons, and ranking without relying on external libraries.

CASE STUDY 08: Tuple-Based Product Catalogue Audit

1. Problem Statement

Audit a product catalogue stored as tuples, detect invalid prices or stock quantities, calculate inventory values, and identify products needing replenishment.

2. Objectives

• Validate each product record.

• Calculate inventory value for valid records.

• Flag stock below the replenishment threshold.

3. Assumptions

• Each tuple contains product ID, product name, unit price, and stock quantity.

• Prices must be greater than zero and stock must be non-negative.

• Products with stock below 10 units need replenishment.

• Inventory value equals unit price multiplied by stock.

4. Algorithm

1. Store catalogue records as tuples.

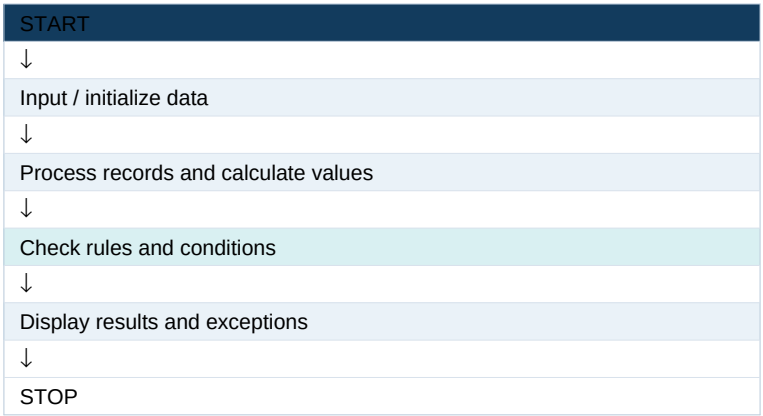
2. Unpack each record and validate price and stock.

3. Calculate inventory value for valid records.

4. Flag low-stock products and invalid records.

5. Display the catalogue audit and total valid inventory value.

5. Flowchart



6. Pseudocode

```
START
Set replenishment threshold = 10
FOR each product tuple: unpack fields
IF price <= 0 OR stock < 0: mark invalid
ELSE: calculate price × stock; add to inventory total
IF stock < 10: flag replenishment
Display valid values, invalid records, and total inventory value
STOP
```

7. Python Source Code

```
# Case Study 8: Tuple-Based Product Catalogue Audit
REPLENISHMENT_THRESHOLD = 10

# (product_id, product_name, unit_price, stock_quantity)
catalogue = [
    ("P101", "Keyboard", 899.00, 18),
    ("P102", "Mouse", 499.00, 7),
    ("P103", "Monitor", 8499.00, 5),
    ("P104", "USB Cable", 0.00, 25),
    ("P105", "Laptop Stand", 1299.00, -2),
```

```
]

total_inventory_value = 0
invalid_records = []
replenishment = []

print("PRODUCT CATALOGUE AUDIT")
print("=" * 52)
for product_id, name, price, stock in catalogue:
    if price <= 0 or stock < 0:
        reason = []
        if price <= 0:
            reason.append("price must be positive")
        if stock < 0:
            reason.append("stock cannot be negative")
        invalid_records.append((product_id, name, reason))
        print(f"INVALID  {product_id} {name}: {', '.join(reason)}")
    else:
        value = price * stock
        total_inventory_value += value
        status = "REORDER" if stock < REPLENISHMENT_THRESHOLD else "OK"
        print(f"{product_id} {name:16} Stock: {stock:3} | Value: Rs.{value:,.2f} | {status}")
        if stock < REPLENISHMENT_THRESHOLD:
            replenishment.append((product_id, name, stock))

print("-" * 52)
print(f"Valid inventory value: Rs.{total_inventory_value:,.2f}")
print("Products needing replenishment:")
for product_id, name, stock in replenishment:
    print(f"  {product_id} - {name} (current stock: {stock})")
print(f"Invalid records: {len(invalid_records)}")
```

8. Test Cases

Test / input	Condition	Expected result
Keyboard	Price 899, stock 18	Valid; inventory value ■16,182; no reorder
Mouse	Price 499, stock 7	Valid; reorder flagged
USB Cable	Price 0	Invalid price
Laptop Stand	Stock -2	Invalid stock quantity

9. Sample Output

```
PRODUCT CATALOGUE AUDIT
====================================================
P101 Keyboard         Stock:  18 | Value: Rs.16,182.00 | OK
P102 Mouse            Stock:   7 | Value: Rs.3,493.00 | REORDER
P103 Monitor          Stock:   5 | Value: Rs.42,495.00 | REORDER
INVALID  P104 USB Cable: price must be positive
INVALID  P105 Laptop Stand: stock cannot be negative
----------------------------------------------------
Valid inventory value: Rs.62,170.00
Products needing replenishment:
  P102 - Mouse (current stock: 7)
  P103 - Monitor (current stock: 5)
Invalid records: 2
```

10. Conclusion

The catalogue audit validates tuple records, computes inventory value, and identifies replenishment needs. It demonstrates tuple unpacking, lists, loops, conditions, and formatted strings.

CASE STUDY 09: Campus Event Financial Planning

1. Problem Statement

Calculate the cost of a campus event, determine participant contributions and sponsorship shortfalls, and estimate the minimum registrations required to break even.

2. Objectives

• Calculate total event costs from several categories.

• Compare costs with sponsorship and registration income.

• Calculate the minimum registration count needed to break even.

3. Assumptions

• Venue ■12,000; equipment ■8,000; catering ■180 per participant; transport ■6,000; certificates ■25 per participant.

• Sponsorship is ■15,000.

• Registration fee is ■350 per participant.

• Break-even registration count is rounded up to the next whole participant.

4. Algorithm

1. Set fixed costs, per-participant costs, sponsorship, and registration fee.

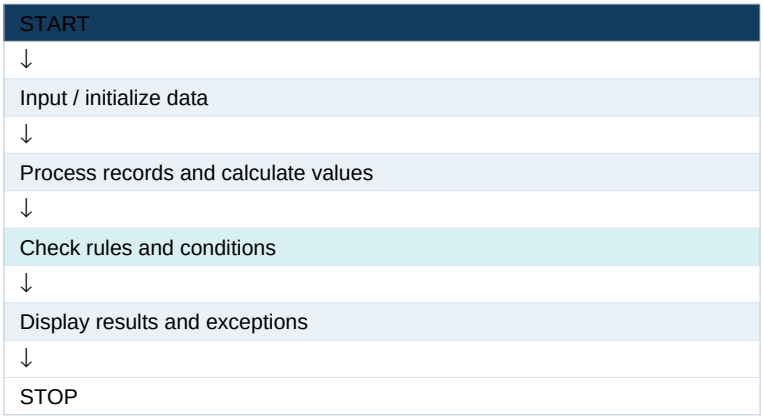
2. Calculate total cost for the planned attendance.

3. Calculate income from sponsorship and registrations.

4. Calculate surplus or shortfall.

5. Calculate minimum registrations needed to cover fixed costs after sponsorship plus variable costs.

5. Flowchart



6. Pseudocode

```
START
Set fixed and variable costs, sponsorship, registration fee, planned participants
Calculate fixed cost and variable cost per participant
Total cost = fixed cost + variable cost × participants
Income = sponsorship + registration fee × participants
Balance = income - total cost
Break-even participants = ceiling((fixed cost - sponsorship) / (fee - variable cost per participant))
Display financial report
STOP
```

7. Python Source Code

```
# Case Study 9: Campus Event Financial Planning
venue_cost = 12000
equipment_cost = 8000
transport_cost = 6000
catering_per_person = 180
certificate_per_person = 25
sponsorship = 15000
registration_fee = 350
planned_participants = 120
```

```
fixed_cost = venue_cost + equipment_cost + transport_cost
variable_cost_per_person = catering_per_person + certificate_per_person
total_cost = fixed_cost + variable_cost_per_person * planned_participants
registration_income = registration_fee * planned_participants
total_income = sponsorship + registration_income
balance = total_income - total_cost

contribution_per_person = registration_fee - variable_cost_per_person
if contribution_per_person <= 0:
    break_even = None
else:
    amount_to_cover = max(fixed_cost - sponsorship, 0)
    break_even = amount_to_cover // contribution_per_person
    if amount_to_cover % contribution_per_person != 0:
        break_even += 1

print("CAMPUS EVENT FINANCIAL PLAN")
print("=" * 42)
print(f"Planned participants : {planned_participants}")
print(f"Fixed costs          : Rs.{fixed_cost:,.2f}")
print(f"Variable cost/person : Rs.{variable_cost_per_person:,.2f}")
print(f"Total event cost     : Rs.{total_cost:,.2f}")
print(f"Sponsorship          : Rs.{sponsorship:,.2f}")
print(f"Registration income  : Rs.{registration_income:,.2f}")
print(f"Total income         : Rs.{total_income:,.2f}")
print(f"Financial balance    : Rs.{balance:,.2f}")
if break_even is None:
    print("Break-even: impossible at the current registration fee.")
else:
    print(f"Minimum break-even registrations: {break_even}")
```

8. Test Cases

Test / input	Condition	Expected result
120 participants	Fee ■350, variable cost ■205/person	Total cost ■50,600; total income ■57,000; surplus ■6,400
Sponsorship ■15,000	Fixed cost ■26,000	■11,000 fixed cost remains to cover
Registration fee <= variable cost	No positive contribution	Break-even reported impossible

9. Sample Output

```
CAMPUS EVENT FINANCIAL PLAN
==========================================
Planned participants : 120
Fixed costs          : Rs.26,000.00
Variable cost/person : Rs.205.00
Total event cost     : Rs.50,600.00
Sponsorship          : Rs.15,000.00
Registration income  : Rs.42,000.00
Total income         : Rs.57,000.00
Financial balance    : Rs.6,400.00
Minimum break-even registrations: 76
```

10. Conclusion

The financial planner estimates event costs, income, surplus or shortfall, and the minimum break-even registration count. It demonstrates arithmetic, conditionals, integer division, lists of inputs, and formatted output.

CASE STUDY 10: University Admission Decision Engine

1. Problem Statement

Evaluate applicants using subject-wise eligibility, course-specific aggregate thresholds, and tie-breaking rules, then produce a ranked list of eligible candidates.

2. Objectives

• Check minimum marks in all subjects.

• Calculate aggregate percentage.

• Apply course-specific minimum aggregate requirements.

• Separate eligible and ineligible applicants.

• Rank eligible applicants using tie-breaking rules.

3. Assumptions

• Subjects are Mathematics, Physics, and Computer Science; each is out of 100.

• Minimum subject mark is 50.

• Course thresholds are CSE = 70%, ECE = 65%, EEE = 60%.

• Ties are resolved by higher Mathematics marks, then higher Computer Science marks, then alphabetical order.

4. Algorithm

1. Store applicant details in a list of tuples.

2. Calculate total and aggregate for each applicant.

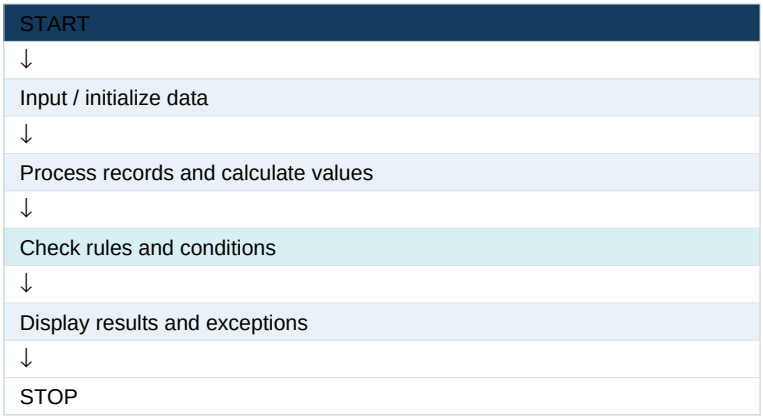
3. Check subject-wise minimum marks and course threshold.

4. Store applicants in eligible or ineligible lists.

5. Rank eligible applicants using nested loops and tie-breaking rules.

6. Display the final ranking and ineligible applicants.

5. Flowchart



6. Pseudocode

```
START
Set minimum subject mark = 50 and course thresholds
Store applicant details
FOR each applicant: calculate aggregate and check subject and course eligibility
Store applicant in eligible or ineligible list
Compare eligible pairs by aggregate, Mathematics, Computer Science, and name
Display ranked eligible applicants and ineligible applicants
STOP
```

7. Python Source Code

```
# Case Study 10: University Admission Decision Engine
course_thresholds = {"CSE": 70, "ECE": 65, "EEE": 60}
minimum_subject_mark = 50
```

```
# (name, course, mathematics, physics, computer_science)
applicants = [
    ("Anil", "CSE", 82, 76, 88),
    ("Bhavya", "ECE", 70, 68, 72),
    ("Charan", "CSE", 74, 70, 76),
    ("Divya", "EEE", 65, 60, 70),
    ("Esha", "CSE", 69, 72, 75),
    ("Farhan", "ECE", 70, 68, 72),
]

eligible = []
ineligible = []

for record in applicants:
    name, course, maths, physics, computer = record
    aggregate = (maths + physics + computer) / 3
    subject_ok = (maths >= minimum_subject_mark and
                  physics >= minimum_subject_mark and
                  computer >= minimum_subject_mark)
    course_ok = aggregate >= course_thresholds[course]

    if subject_ok and course_ok:
        eligible.append((name, course, maths, physics, computer, aggregate))
    else:
        reasons = []
        if not subject_ok:
            reasons.append("one or more subjects below 50")
        if not course_ok:
            reasons.append(f"aggregate below {course_thresholds[course]}%")
        ineligible.append((name, course, aggregate, reasons))

# Rank by aggregate descending, maths descending, computer science descending,
# then name ascending. Nested loops keep this within introductory syllabus topics.
for i in range(len(eligible)):
    for j in range(i + 1, len(eligible)):
        first = eligible[i]
        second = eligible[j]
        swap = False
        if first[5] < second[5]:
            swap = True
        elif first[5] == second[5] and first[2] < second[2]:
            swap = True
        elif (first[5] == second[5] and first[2] == second[2]
              and first[4] < second[4]):
            swap = True
        elif (first[5] == second[5] and first[2] == second[2]
              and first[4] == second[4] and first[0] > second[0]):
            swap = True
        if swap:
            eligible[i], eligible[j] = eligible[j], eligible[i]

print("UNIVERSITY ADMISSION RANKING")
print("=" * 42)
for rank, record in enumerate(eligible, start=1):
    name, course, maths, physics, computer, aggregate = record
    print(f"Rank {rank}: {name} | Course: {course} | Aggregate: {aggregate:.2f}%")
print("\nINELIGIBLE APPLICANTS")
for name, course, aggregate, reasons in ineligible:
    print(f"{name} | Course: {course} | Aggregate: {aggregate:.2f}% | {', '.join(reasons)}")
```

8. Test Cases

Test / input	Condition	Expected result
Anil	CSE; marks 82,76,88	Eligible; aggregate 82.00%; rank 1
Bhavya	ECE; marks 70,68,72	Eligible; aggregate 70.00%
Esha	CSE; marks 69,72,75	Eligible: aggregate 72.00% and all subjects >=50; no rejection
Tied applicants	Bhavya and Farhan have identical marks	Alphabetical tie-break places Bhavya before Farhan

9. Sample Output

```
UNIVERSITY ADMISSION RANKING
==========================================
Rank 1: Anil | Course: CSE | Aggregate: 82.00%
Rank 2: Charan | Course: CSE | Aggregate: 73.33%
Rank 3: Esha | Course: CSE | Aggregate: 72.00%
Rank 4: Bhavya | Course: ECE | Aggregate: 70.00%
Rank 5: Farhan | Course: ECE | Aggregate: 70.00%
Rank 6: Divya | Course: EEE | Aggregate: 65.00%

INELIGIBLE APPLICANTS
```

10. Conclusion

The admission engine checks course and subject requirements, records reasons for rejection, and applies deterministic tie-breaks. It demonstrates conditions, loops, lists, tuples, tuple unpacking, and ranking.